

Homework associated to L03 Design a mode

The ultimate goal of modelling is to perform simulations to be analysed in order to gain domain knowledge.

The first step is to design a suitable mathematical model and this can be done by means of rewriting rules, but to make them interpretable by a machine, it is necessary to define a syntax and accompany it with a parser able to interpret it and to pass the information to a simulator.

model definition file syntax

Here follows the description of the syntax rules needed to write a file containing the model description. The file can be parsed by means of the functions contained in the library `modelParser.py`.

The file naming does not follow any particular rule.

To properly define a model, the file should contain four sections: modelling framework, rewriting rules, initial conditions and feed species. Each section is identified by a tag or a couple of tags enclosing it.

modelling framework

Kind> Stoc declare a stochastic description of the model (copy number, stochastic constants)

Kind> Det declare a deterministic description of the model (concentration, kinetic constants)

rewriting rules

The section starts at the line marked by the tag `Rules>` and terminates with a line marked by the tag `<Rules`.

Each rewriting rule is on a single line and the character `;` separate the rule from the rate constant (a real value).

The accepted syntax for the rule is:

```
integer * Species + ... + integer * Species -> integer * Species + ... + integer * Species ; rateConstant
```

It is possible to use the word “lambda” or leave an empty side to mean no object.

```
Rules>
A -> B; 1
A + B -> 2*C; 5e-3
A -> ; 1.07
<Rules
```

Initial conditions

The section starts at the line marked by the tag `X0>` and terminates with a line marked by the tag `<X0`.

The initial amount/concentration of each species is declared on a separate line with the following syntax:

```
A = amount
```

Species with a not set value are initialized with 0.

```
X0>
A = 1000
B = 0
<X0
```

Feed species

When it is necessary have species with a fixed amount/concentration during the simulation it is necessary to insert them in this section.

The section starts at the line marked by the tag `XFeed>` and terminates with a line marked by the tag `<XFeed`. Each line within the tags must contain only the symbol of the species to be kept constant.

```
XFeed>
A
B
<XFeed
```

Commented example

Below there is a commented example of the content of a model file:

```
Kind> Stoc # Stoc = stochastic / Det = deterministic
Rules> # here start the rewriting rules declaration; one on each line;
A -> B; 1 # one rule each line with the ";" to separate the rate constant
2*B -> C; 5e-3 # the accepted syntax is: integer * Species + ... + integer * Species -> integer * Species + ...
C -> ; 0.009 # the word "lambda" or an empty side means "nothing" (C goes to nothing)
<Rules # here ends the rewriting rules declaration

X0> # here start the initial species concentration/copy number declaration
A = 1000 # one species each line. Undeclared Species are initialized with 0
B = 0 #
<X0 # here ends the Species initialization

XFeed> # here starts the declaration for the feeding species. One per line, only the symbol
A # this means that the species A will be kept at a constat value during the simulations
<XFeed # here ends the declaration of the feeding species
```

Template

```
Kind>
Rules>
<Rules
```

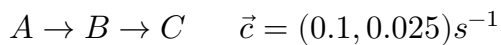
```
X0>
<X0
```

```
XFeed>
<XFeed
```

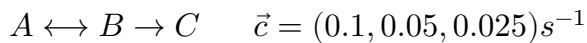
Exercise:

According to the previously defined syntax, write the model definition file for the following models:

- **Cascade**



- **Cascade with equilibrium**



- **Michaelis Menten**



Exercise: Prokaryotic auto-regulatory gene network

from [Wang et al. BMC Systems Biology 2010, 4:99](#)

The model we tested is a prokaryotic auto-regulatory gene network in which dimmers of a protein repress its own gene transcription by binding to a regulatory region upstream of the gene. The system, involving both transcription and translation, can serve as a simple, yet illustrative, example of gene regulation [31,39].

The reactions in the network are given below:

R1 : $\text{DNA} + \text{P2} \rightarrow \text{DNA.P2}$,
R2 : $\text{DNA.P2} \rightarrow \text{DNA} + \text{P2}$,
R3 : $\text{DNA} \rightarrow \text{DNA} + \text{mRNA}$,
R4 : $\text{mRNA} \rightarrow \text{.}$,
R5 : $2\text{P} \rightarrow \text{P2}$,
R6 : $\text{P2} \rightarrow 2\text{P}$,
R7 : $\text{mRNA} \rightarrow \text{mRNA} + \text{P}$,
R8 : $\text{P} \rightarrow \text{.}$

Here DNA, P, P2 and mRNA represent promoter sequences, proteins, protein dimmers and messenger RNA respectively. In this model, mRNAs and proteins are synthesized by transcription and translation processes (R3 and R7), and destroyed by degradation (R4 and R8). The proteins can form a dimer P2 (R5 and R6), which binds and unbinds to DNA (R1 and R2). When a protein dimer binds to the promoter, it represses mRNA production. Overall, the network implements a self-regulatory mechanism to control the synthesis of the protein product, suppressing the transcription when the protein product is abundant. Note that $\text{DNA} = \text{DNA} + \text{DNA.P2}$ is a conserved quantity in the system.

... within a time window of [0 50) with $(k_1, \dots, k_8) = (0.1, 0.7, 0.35, 0.3, 0.1, 0.9, 0.2, 0.1)$.

And $\text{DNA} = 2$

Assignment

Encode in a model file the above described model by means of rewriting rules and according to the syntax explained in the first part of this document.

Exercise: the repressilator

from [Elowitz, M., Leibler, S. A synthetic oscillatory network of transcriptional regulators. Nature 403, 335–338 \(2000\)](#)

... We used three transcriptional repressor systems that are not part of any natural biological clock to build an oscillating network, termed the repressilator, in *Escherichia coli*. The network periodically induces the synthesis of green fluorescent protein as a readout of its state in individual cells.

Network construction

The three repressors (*lacI*, *tetR* and *cl*), the λ promoter P_R and the unstable variants of *gfp* were all cloned by the polymerase chain reaction (PCR) and verified by sequencing. In the case of *lacI*, the GTG start codon was changed to ATG with the 5' primer. 39' PCR primers were used to add the coding sequence for the 11-amino-acid *ssrA* tag to the three repressors. Tagging of full-length λ repressor, as well as *lacI* and *tetR*, resulted in functional proteins that required increased induction to achieve full repression, consistent with decreased stability. Promoter sequences P_{LlacO1} and P_{LtetO1} were obtained from *pZE21-MCS1* and *pZE12-luc*. The repressilator plasmid was constructed in three successive cloning steps from intermediate plasmids containing the three transcriptional units. The reporter plasmid was constructed by cloning the *GFPaav* variant, which has a half-life of around 90 min, into *pZE21-MCS1* (ref. 6). The finite rate at which *GFP* is oxidized to its fluorescent form and its relatively long half-life limit our ability to observe fast dynamic phenomena. The presence of additional *tet* operators on the reporter plasmid constitutes a perturbation to the single-plasmid repressilator network by titrating out repressors.

Assignment

Exploiting the auto-regulatory gene network of the preceding exercise, build the model for the repressilator, and write it in a model definition file.

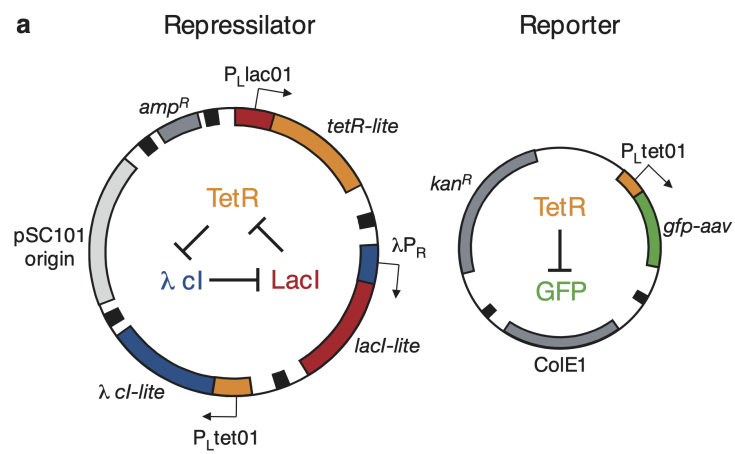


Figure 1: The model